# NORDIC SHOPPING

## Cloud Transformation to Microsoft Azure

*Complete Project Explainer — Architecture, Decisions & Build Order*

Consolidated from the project's current document set

Prepared: 4 August 2026

Owner: Amin Azad

West Europe (active) · Sweden Central (warm standby)
Planning baseline DKK 15,000/month · Authorized envelope DKK 16,500/month

# Table of Contents

# 1. Executive Summary — Why This Project Exists

Nordic Shopping is a Copenhagen-based e-commerce marketplace: about 35 employees, 40,000 registered customers, 150 active vendors, and roughly 600 orders a day. Everything today — the customer website, the API, the vendor portal, the admin portal, the SQL database and the file storage — runs from a single on-premises location. That single location is the entire problem this project solves.

If that one site loses power, network connectivity, or suffers a hardware failure, every channel goes down at once: customers can't shop, vendors can't fulfil orders, and staff can't administer the platform. Scaling up requires buying and racking new hardware. Releases are mostly manual, which makes them slow and risky. Monitoring is fragmented, so problems are often discovered by customers before they're discovered by the team. Access control is not consistently governed. And there is no proven, tested way to recover the business in another location.

The company also wants to grow: from Denmark only to Denmark, Sweden and Norway; from 40,000 customers to a three-year target of 250,000; from 600 daily orders to 5,000; and from monthly releases to weekly or on-demand releases. The current environment cannot support that growth safely.

## 1.1 The decision in one sentence

Nordic Shopping will re-platform (not rewrite) its four applications onto managed Azure PaaS services, running actively in West Europe with a warm standby in Sweden Central, behind a single protected entry point (Azure Front Door), with all data services made private, all infrastructure defined as code, and a disciplined 16-week delivery plan that only allows production traffic once each risk area has passed a testable gate.

## 1.2 What we are deliberately NOT doing (yet)

A recurring theme throughout this project is buying only the capability the business needs right now, while leaving a clearly documented, triggered path to buy more capability later. We are explicitly NOT doing, in Release 1:

- Rewriting the application as microservices — it stays a modular monolith behind one API.
- Running active-active across two regions — West Europe is active, Sweden Central is a warm (but not live-serving) standby.
- Buying Front Door Premium, API Management, Service Bus, Redis, Kubernetes (AKS), or virtual machines.
- Deploying Microsoft Sentinel or a 24/7 staffed Security Operations Centre.
- Giving the AI Operations Assistant any ability to change, deploy, or remediate anything — it is strictly read-only and advisory.
- Assuming Release 1's sizing will still be correct at 250,000 customers — growth requires a new, evidence-based capacity review.

Every one of these has an explicit “trigger” documented later in this report (see Section 7 and Section 12) — a measurable condition under which the team should revisit the decision. Nothing is deferred forever without a plan; nothing is added just because it exists.

## 1.3 Headline numbers

| Measure | Value |
|---|---|
| Company profile | 35 employees · 40,000 customers · 150 vendors · ~600 orders/day |
| Three-year target | 80 employees · 250,000 customers · 800 vendors · 5,000 orders/day · DK+SE+NO |
| Active region | West Europe (westeurope) |

| Measure | Value |
|---|---|
| DR region | Sweden Central (swedencentral) — warm standby |
| Availability target | ≥ 99.9% monthly for customer-facing services |
| Recovery objectives | RTO ≤ 60 minutes · RPO ≤ 15 minutes |
| Normal-month planning baseline | DKK 15,000/month (excl. VAT) |
| Authorized monthly envelope | DKK 16,500/month (excl. VAT) |
| One-time migration/delivery allowance | DKK 14,000–38,000 |
| Delivery timeline | 16 weeks, 9 gated phases, after approval |

## 2. How the Project Documentation Is Organized

This project is planned as ten linked documents plus a set of nine architecture diagrams, dated 4 August 2026. Nothing in this report introduces a different design — it is a single, readable walk-through of that approved document set, reorganized so you can read the reasoning in one place before any code is written.

| # | Source document | What it answers |
|---|---|---|
| 01 | Business Case | Why are we doing this, and what is the investment ask? |
| 02 | Business Requirements | What must the system do, and how will we prove it does it? |
| 03 | Current Environment | What do we have today, and what's wrong with it? |
| 04 | Target Architecture | Exactly what will we build? |
| 05 | Migration Strategy | How do we get from today's environment to Azure without breaking the business? |
| 06 | Cost Estimation | What will it cost now, and what will it cost as we grow? |
| 07 | Security Assessment | What are the risks, and are they acceptable? |
| 08 | Security Strategy | Exactly which technical controls close those risks? |
| 09 | Project Roadmap | In what order do we build it, and what proves each step is done? |
| 10 | Architecture Decision Records (ADRs) | Why this choice and not another, for every major decision? |

# 3. Where We're Starting From (Current Environment)

Before any Azure decision makes sense, it helps to see the shape of what exists today. Nordic Shopping's current platform is a Windows-based, single-site environment: four logical applications (Customer Web, Nordic API, Vendor Portal, Admin Portal), a Microsoft SQL Server database, shared file storage, Active Directory for employee identity, and a local backup server — all in one physical location.

## 3.1 The four applications, then and now

The system is treated as a modular monolith: one API is the single gateway to business logic and data, and three front-ends (customer web, vendor portal, admin portal) plus the existing mobile apps all call that API rather than touching the database directly. This shape doesn't change during the migration — we are re-platforming these four applications onto Azure App Service, not rewriting them into microservices.

## 3.2 What's materially wrong with the current setup

- Single failure domain — a site, power, network, or hardware failure takes down every channel at once, and there is no tested regional recovery process.
- Manual, hardware-bound scaling — capacity increases need procurement and manual configuration, with long lead times.
- Manual deployment — application and infrastructure changes are largely manual, which creates inconsistent environments and unsafe releases.
- Fragmented monitoring — there is no centralized view connecting customer requests, application failures, database health, provider failures, deployments and security events, so incidents are detected late.
- Inconsistent identity and access governance — MFA, privileged access, lifecycle management and role separation all need validation; some may not exist at all.
- Backups exist, but there's no evidenced regional recovery — the backup server is in the same site, so it doesn't protect against losing the site itself.
- Team time goes to server maintenance instead of commerce features.
- No proven path to Nordic expansion (Sweden, Norway) at 5,000 orders/day.

## 3.3 A deliberately honest gap: what we don't know yet

The Current Environment document is explicit that many low-level facts — exact server counts, SQL Server edition, network topology, firewall rules, exact file volumes — are not yet confirmed. Rather than guess, the project defines a 12-item Discovery Register (server/app inventory, network/DNS map, identity inventory, SQL assessment, file inventory, dependency assessment, provider register, monitoring/backup evidence, security gap validation, current cost baseline, blackout dates, and data-protection obligations) that must be completed, with a named owner and exit condition for each item, before build work starts in earnest. This matters for how you read the rest of this report: sizing decisions (like “2 vCore SQL” or “P1v3 App Service”) are testable starting hypotheses, not proven facts — they get validated by real discovery data and load tests before go-live, not assumed to be right from day one.

# 4. The Business Case — Why We're Spending the Money

## 4.1 How the numbers evolved (and why that's a good sign, not a red flag)

It's worth understanding the actual history here, because it explains why the budget moved. The project did not start with a fully-priced architecture and then write a business case to match it — it ran the other way round, like a normal planning funnel:

| Stage | When | What happened |
|---|---|---|
| Initial business case (V1) | 26 May 2026 | Order-of-magnitude ask, approved before any Azure design existed. Authorized discovery under a placeholder DKK 10,000/month ceiling — a guess, not an engineered estimate. |
| Discovery & requirements | 2 Jun – 9 Jul 2026 | Captured what the platform actually needs to do and what it looks like today. |
| Target architecture drafted | 15 Jun – 3 Aug 2026 | The real Azure design was built, tested against requirements, and iterated. |
| First real cost estimate | 24 Jul 2026 | Once a concrete architecture existed, it could finally be priced accurately — and the old DKK 10,000 ceiling (and an even earlier DKK 6,700–7,200 guess) turned out to be understated. |
| Migration, security & roadmap finalized | 23 Jul – 1 Aug 2026 | Delivery plan, controls and cutover approach locked against the stabilized architecture. |
| 16 ADRs consolidated | 3 Aug 2026 | Every accepted design choice and its rejected alternatives written down formally. |
| This business case (current baseline) | 4 Aug 2026 | Restates the investment ask using the resulting architecture and cost model as evidence, and adds two late resilience decisions found during final review. |

In other words: the DKK 15,000/16,500 figures aren't a number picked to sound reasonable — they're what the approved, diagrammed, testable design actually costs, priced line-by-line. That's a more trustworthy number than an early guess, even though it's higher.

## 4.2 Business objectives this project must hit

1. Reduce dependency on one physical location.
2. Establish tested recovery: end-to-end RTO ≤ 60 minutes, RPO ≤ 15 minutes.
3. Achieve ≥ 99.9% monthly availability for customer-facing services after go-live.
4. Support current load and provide a measurable path toward 250,000 customers / 5,000 daily orders.
5. Protect customer, vendor and company data through strong identity, least privilege, private data access and auditable controls.
6. Make infrastructure and application delivery repeatable through Bicep and GitHub Actions using OIDC.
7. Enable weekly or on-demand releases with controlled rollback and minimal customer interruption.
8. Improve incident detection and diagnosis through centralized telemetry, dashboards and actionable alerts.
9. Keep normal Azure consumption within the authorized DKK 16,500/month envelope.
10. Introduce an employee-only, read-only AI Operations Assistant — but only after the monitoring platform itself passes acceptance.

> *These are target outcomes, not achieved facts. Each one only counts as "done" once there's test evidence behind it — that discipline runs through every document in this project.*

## 4.3 Options that were considered and rejected

| Option | Assessment | Decision |
|---|---|---|
| Continue on-premises | Lowest short-term change, but keeps every current risk (single site, scaling, delivery, recovery) | Rejected |
| Lift-and-shift to Azure VMs | Removes the physical-site dependency, but you still patch, cluster and manage servers yourself | Rejected |
| AKS + microservices redesign | Powerful, but adds cost, skills demand and delivery complexity beyond current need | Rejected for Release 1 |
| Active-active multi-region | Stronger continuous service, but a big jump in data-consistency, testing and operating complexity | Deferred |
| Managed Azure PaaS with warm standby | Balances resilience, security, delivery capability, operating effort and cost | Approved — this is the design |

## 4.4 The financial ask, precisely

| Line | Amount | Meaning |
|---|---|---|
| Azure service-consumption subtotal | DKK 13,950/month | What the priced architecture actually consumes |
| Planning contingency | DKK 1,050/month (~7.5%) | Usage, telemetry, currency and estimation variance |
| Normal-operation planning baseline | DKK 15,000/month | The number to plan against |
| Authorized operating envelope | DKK 16,500/month | Maximum normal spend without further approval |
| Controlled headroom | DKK 1,500/month | Buffer for moderate traffic/telemetry variance — not a target to spend |
| Annual planning baseline | DKK 180,000/year | 12 × DKK 15,000 |
| Annual authorized envelope | DKK 198,000/year | 12 × DKK 16,500 |
| One-time migration/delivery allowance | DKK 14,000–38,000 | Rehearsals, testing, parallel operation, delivery consumption — separate from the recurring budget |

These figures exclude VAT, salaries, external implementation labour, payment-provider charges, Microsoft 365 licensing and other third-party business services — they cover Azure consumption only, and must be refreshed in the Azure Pricing Calculator before procurement and before production cutover.

Approving this business case authorizes the architecture, the DKK 15,000 planning baseline, spend up to DKK 16,500/month without further approval, and the DKK 14,000–38,000 one-time delivery allowance. It explicitly does NOT authorize production cutover — cutover is a separate, later decision that requires objective evidence across security, performance, migration, DR, operations and cost (see Section 13).

# 5. What the System Must Do (Requirements at a Glance)

The requirements document uses precise language on purpose: “Shall” means required for Release 1 acceptance; “Target” means a measurable objective validated before it becomes a commitment; “Deferred” means explicitly excluded unless a change-control process approves it later. Choosing an Azure service never counts as satisfying a requirement by itself — there must be test, configuration or monitoring evidence behind every one.

## 5.1 Requirement categories and what they cover

| Category | ID prefix | Sample of what's required |
|---|---|---|
| Business | BR | Continue shopping/checkout/tracking during migration; vendors see only their own data; AI stays advisory-only |
| Functional | FR | Web/mobile/portals call only the API; API is the sole boundary for data access, logic and auditing; uploads go through quarantine + malware scan |
| Availability, reliability, recovery | NFR-AVL / NFR-REL / NFR-DR | ≥99.9% monthly availability; ≥2 active workers in WEU; RTO ≤60 min; RPO ≤15 min; CI/CD cannot activate DR |
| Performance & scalability | NFR-PER / NFR-SCL | Pages usable in ≤3s (p95); checkout API ≤1s (p95); WEU autoscale 2–4 workers; SWC holds ≥2 standing workers |
| Identity & authorization | SEC-IAM | Entra External ID for customers; B2B for vendors; workforce Entra ID + MFA for staff; every request is server-side authorized |
| App, network, data security | SEC-APP / SEC-NET / SEC-DAT | Front Door is the only ingress; SQL/Blob/Key Vault/OpenAI have public access disabled; encryption in transit and at rest |
| Privacy & compliance | CMP | GDPR principles; masked non-production data; no full card data stored; documented breach procedure |
| Monitoring & operations | OPS | Centralized telemetry; alert catalogue with named owners; dashboards for health, dependencies, recovery state, cost |
| Delivery & change | DEV | Bicep IaC; GitHub Actions with OIDC (no stored secrets); staging slots + human approval before production swap |
| Migration | MIG | Full inventory before build approval; ≥ 2 rehearsals; reconciled data; controlled cutover with rollback |
| Cost & governance | FIN | Stay within the DKK 16,500 envelope or get formal approval; tagged resources; monthly cost review |
| AI Operations Assistant | AI | Only released after monitoring is accepted; read-only; redacted inputs; audited; cannot affect commerce if it fails |

## 5.2 Production acceptance gates

Production cutover cannot occur until every one of these gates has evidence and a named approver:

| Gate | Required evidence |
|---|---|
| Architecture | Bicep deployment matches the approved architecture and accepted ADRs |
| Application | Functional, integration, UAT, performance and rollback tests pass |
| Identity / security | Role/isolation tests, attack tests and risk closure accepted |
| Data / migration | Two rehearsals, reconciliation and rollback evidence accepted |

| Gate | Required evidence |
|---|---|
| Recovery | Restore, regional failover and failback meet RTO/RPO |
| Operations | Monitoring, alerts, runbooks, ownership and responder routing accepted |
| Cost | Refreshed forecast within DKK 16,500 or extra funding approved |
| Cutover | Entry criteria, communications, go/no-go and rollback authority signed |

# 6. The Target Architecture — Exactly What Gets Built

This is the heart of the project: the single technical design that every other document supports. It is a managed Azure PaaS modular monolith, where the Nordic API remains the one and only production business-logic and data-access boundary.

## 6.1 Fixed implementation decisions, at a glance

| Area | Approved decision |
|---|---|
| Architecture style | Managed PaaS, modular monolith, API-owned business logic |
| Regional model | Active–passive: West Europe active, Sweden Central warm standby |
| Public ingress | Azure Front Door Standard — one route + origin group per public app |
| Edge protection | Front Door custom WAF match/rate-limit rules + origin access restrictions |
| Applications | Customer Web, Nordic API, Vendor Portal, Admin Portal — separate Linux App Services |
| Mobile | Existing apps stay API clients; not hosted as Azure workloads |
| Compute SKU | Linux App Service Premium v3, P1v3 |
| Compute count | WEU: 2–4 instances (autoscale) · SWC: 2 standing standby instances (no scale-up delay before DR) |
| Deployments | One staging slot per app per region; slot swap after validation |
| Database | Azure SQL Database General Purpose, 2 vCores provisioned, zone-redundant primary, LRS backups |
| Database DR | Failover group to an equal-size secondary; manual promotion initially |
| Object data | Private StorageV2 per region; LRS baseline + replication for selected critical blobs |
| Secrets | One private Key Vault per region; RBAC, soft delete, purge protection |
| Customer identity | Microsoft Entra External ID external tenant |
| Vendor identity | Entra B2B guest users in the workforce tenant |
| Employee identity | Workforce Entra ID, MFA, Conditional Access |
| Workload identity | System-assigned managed identity for every App Service |
| Network | One VNet per region, delegated integration subnet, private-endpoint subnet, Private DNS |
| Delivery | Bicep + GitHub Actions with OIDC workload federation |
| Observability | Application Insights (workspace-based), Log Analytics, Azure Monitor alerts, workbooks |
| AI | Read-only AI Operations Assistant, released after monitoring acceptance, no autonomous actions |

> *Explicitly not in Release 1: Front Door Premium, API Management, Service Bus, Redis, Kubernetes, virtual machines, Application Gateway, Azure Load Balancer, Azure AI Search, active-active database writes, customer-facing generative AI, and direct public uploads.*

## 6.2 Architecture principles that shaped every decision

- Managed identities replace access keys and client secrets wherever Azure supports it.
- SQL, Storage and Key Vault all have public network access disabled — no exceptions.
- Only the API touches transactional data; portals and mobile clients never connect directly.

- Applications stay independently deployable even while they share regional compute.
- Production always starts with at least two active App Service workers.
- The same regional Bicep modules build both the primary and the DR infrastructure — no hand-built DR environment.
- Every release is backward compatible, observable, and reversible.
- Replication, high availability and backup are treated as three separate controls, not one.
- Retriable business operations (orders, payments) use idempotency keys and reconciliation.
- Production changes and disaster declarations always require a named human's approval — never an automated pipeline.

## 6.3 End-to-end topology

Customers, vendors and employees authenticate through the appropriate Entra identity system, then reach the platform through Azure Front Door — the single public Layer-7 entry point that also performs inter-region routing. In West Europe, a P1v3 App Service plan runs 2–4 workers hosting all four apps, each with a private path to SQL, Blob and Key Vault. Sweden Central runs a matching P1v3 plan with 2 standing workers and its own private SQL secondary, Blob and Key Vault — kept warm and current, but not serving live traffic until a human authorizes DR. No separate Azure Load Balancer is needed, because App Service distributes requests across workers within each plan and there's no public VM pool or Layer-4 workload in this design.

## 6.4 Networking and private access

| Region | VNet | Integration subnet | Private-endpoint subnet | Reserved future subnet |
|---|---|---|---|---|
| West Europe | 10.10.0.0/16 | 10.10.1.0/24 | 10.10.2.0/24 | 10.10.3.0/24 |
| Sweden Central | 10.20.0.0/16 | 10.20.1.0/24 | 10.20.2.0/24 | 10.20.3.0/24 |

West Europe links four Private DNS zones (SQL, Blob, Key Vault, and Azure OpenAI); Sweden Central links three (SQL, Blob, Key Vault — no Azure OpenAI, because there's no AI resource deployed in the DR region in Release 1). Public network access on SQL, Blob and Key Vault is only disabled after private resolution and identity access tests pass — sequencing here really matters, since disabling public access before private paths work would take the platform offline.

A key architectural point worth internalizing: private connectivity and authorization are two separate controls. A private endpoint proves the request can reach the service network-wise; it says nothing about whether that caller is allowed to read or write that data. Both checks are required on every request — this shows up repeatedly across the security and identity sections below.

## 6.5 Front Door, routing and WAF

| Custom domain | Route | Primary origin (priority 1) | DR origin (priority 2) |
|---|---|---|---|
| www.nordicshopping.dk | route-web | WEU Customer Web | SWC Customer Web |
| api.nordicshopping.dk | route-api | WEU Nordic API | SWC Nordic API |
| vendor.nordicshopping.dk | route-vendor | WEU Vendor Portal | SWC Vendor Portal |
| admin.nordicshopping.dk | route-admin | WEU Admin Portal | SWC Admin Portal |

Each app gets its own origin group, so one failed portal doesn't drag unrelated applications into failover. Health probes hit /health/ready every 30 seconds; an origin only returns 200 when its mandatory regional dependencies are actually ready. Crucially, the DR API origin stays disabled or unhealthy for business traffic until the DR database has been promoted and a human incident commander authorizes recovery — this stops Front Door from ever routing writes to an unprepared secondary database.

The WAF policy runs in detection mode while thresholds are tuned against real traffic, then moves to prevention before go-live. It uses custom rules (emergency IP block, admin geo-restriction, blocked HTTP methods, request-size limits, and tiered rate limits for anonymous API traffic, auth endpoints and admin endpoints) rather than the Premium tier's managed rule sets — a deliberate cost trade-off, compensated for by strict input validation, origin lockdown, and testing (see Section 8).

## 6.6 Compute and application hosting

Both regions run Linux App Service on P1v3 plans, one plan per region shared by all four applications and their staging slots. West Europe autoscales between 2 and 4 workers (add one worker when average CPU is above 70% for 10 minutes; remove one when below 35% for 20 minutes, with conservative cooldowns to avoid oscillation). Sweden Central holds a standing minimum of 2 workers at all times — not scaled up only when a disaster is declared — so DR traffic can be served immediately, with no scale-up delay in the critical recovery path. Both plans are configured zoneRedundant: true, since two instances is exactly the minimum Premium v3 requires for zone redundancy; Azure spreads the existing instances across availability zones at no extra per-instance cost.

All four apps and their slots share the plan's workers, CPU and memory — staging is not free capacity, and this sharing is a conscious cost/isolation trade-off with a documented trigger: the API moves to its own dedicated P1v3 plan in both regions once it repeatedly exceeds 60% of shared CPU/memory, or causes latency for other apps, or needs independent scaling or security isolation. That move happens through a new ADR and a cost approval — never improvised mid-incident.

## 6.7 The application itself — module boundaries

| Module | Responsibility | Key consistency rule |
|---|---|---|
| Identity/Profile | App user/vendor mapping and preferences | Entra subject is the immutable external key |
| Catalogue | Products, categories, media metadata, prices | A vendor can modify only their own catalogue |
| Inventory | Available/reserved quantities | Reservations are transactional and expire |
| Cart/Checkout | Cart validation, totals, order initiation | Server always recalculates price and availability |
| Orders | Order state machine and history | Transitions are validated and auditable |
| Payments | Provider intent/reference and status | No card data stored; webhook + command idempotency |
| Fulfilment | Vendor acceptance, shipment, delivery events | Vendor ownership enforced on every action |
| Notifications | Email/SMS/push requests and delivery status | A notification failure never rolls back a committed order |
| Administration | Support actions, refunds, reports | High-risk changes require role checks and an audit event |

Controllers never touch SQL directly — they call application services, and all data access is parameterized (no string-built queries, ever). Cross-module workflows that are short and local use one SQL transaction; calls that reach outside the database (to payment/delivery/notification providers) go through a transactional outbox instead (see Section 6.9).

## 6.8 Database

Azure SQL Database, General Purpose tier, 2 vCores provisioned in each region. The West Europe primary is zone-redundant (a V6 addition — see Section 7); the Sweden Central secondary is an equal-size geo-secondary joined to the primary via a failover group, so the application always connects to the failover-group listener name, never a region-specific server name. Automatic failover is deliberately disabled (Manual policy) in Release 1, because promoting the secondary can mean accepting some data loss, and that decision needs a human, not an

automated trigger. Database changes follow expand/migrate/contract: add compatible schema → deploy code that understands both the old and new shape → migrate the data → remove the old structure in a later release. Destructive same-release migrations are prohibited.

## 6.9 Reliable integration with external providers

Payment, delivery, email and push providers get explicit connection/read deadlines, bounded exponential backoff with jitter, circuit breakers, correlation IDs and idempotency keys. Inbound webhooks are verified for TLS, provider signature, timestamp/replay window, and de-duplicated by event ID.

Release 1 deliberately uses a SQL transactional outbox rather than a message broker:

1. Business state and an outbox event commit together, in one SQL transaction.
2. A background worker claims outbox records with a lease.
3. Delivery status, attempt count, next-attempt time and error class are recorded.
4. Records that keep failing move to a review state and alert operations.
5. A reconciliation job compares Nordic Shopping's state against the provider's state.

Azure Service Bus becomes mandatory only once measured backlog, throughput, multiple independent consumers, or complex retry semantics exceed what this simpler design can handle — see ADR-012 in Section 7.

## 6.10 The AI Operations Assistant

This is a Release 1 feature, but a tightly bounded one. It lives only inside the Admin Portal, on an incident record, and does exactly four things: summarizes an alert and its context; correlates approved, redacted telemetry with recent deployments; retrieves relevant excerpts from version-controlled runbooks; and proposes ranked investigation steps with evidence links and stated uncertainty. It is not a customer chatbot, cannot diagnose from unrestricted production data, cannot talk to customers, cannot approve changes, and cannot remediate anything.

Technically: the Admin Portal calls the API, which re-checks the employee's token, application role, incident access and per-user quota, then builds a server-side context package — the browser never supplies an arbitrary data query or system prompt. Only one private Azure OpenAI resource exists, in West Europe, reached over Private Link; the API's managed identity gets only Cognitive Services OpenAI User, and API keys are disabled entirely. Every answer must return a summary, evidence references, suggested checks, a confidence/limitations statement, and humanApprovalRequired=true. The feature is intentionally unavailable during a West Europe regional outage — core shopping, administration and DR all keep working without it, avoiding the cost and complexity of a second AI deployment for a non-critical capability.

Before this feature is ever turned on for real incidents, a 30-case evaluation set must pass: 100% refusal of write/remediation requests, zero successful prompt-injection overrides, zero secrets or prohibited personal data in inputs/outputs, every factual claim linked to real evidence, at least 80% of investigation steps rated useful by two reviewers, complete audit trails, working quota enforcement, and a tested kill switch. No Sev 1/2 incident workflow is allowed to depend on it.

## 6.11 Cost and capacity model, structurally

The architecture document deliberately fixes exact quantities and SKUs so the cost estimate prices the real design, not an approximation of it: 4 P1v3 worker-equivalents (2 WEU + 2 SWC standing standby), 1 zone-redundant WEU SQL primary + 1 like-for-like SWC secondary (2 vCore each), Front Door Standard + WAF, 2 Storage accounts with replication, 7 private endpoints across 4 unique zone types, 2 Key Vaults, Log Analytics + Application Insights, Defender for Storage malware scanning, and the bounded AI Operations Assistant. See Section 10 for the full cost breakdown.

# 7. Why We Chose What We Chose — The 16 Architecture Decisions

This is arguably the most important section for you to read before writing code, because it captures the reasoning — not just the outcome. Each decision below was formally recorded (ADR-001 through ADR-016), with the alternatives that were seriously considered and rejected, the trade-offs the team knowingly accepted, and the measurable condition ("review trigger") that would cause the team to revisit it. A new ADR is required whenever a proposal changes an accepted service, region, trust boundary, or data-access path; changes RPO/RTO/availability/security posture; enables something deferred; increases the approved budget; adds a new public endpoint or sensitive-data processor; gives AI or automation permission to touch production; or creates a material new operational skill requirement.

## ADR-001 — Azure Front Door Standard (not Premium)

**Why this decision was needed:**

We need one global HTTPS entry point with routing, health probes and failover, plus edge filtering — at a cost proportionate to current scale.

**Alternatives considered and their fate:**

- Application Gateway — rejected: regional, not the global entry + failover we need.
- Azure Load Balancer — rejected: Layer 4; unnecessary with no public VM pool.
- Direct App Service exposure — rejected: inconsistent entry points, allows edge-control bypass.
- Front Door Premium — deferred: higher base cost; custom rules + origin restrictions cover current risk.

**Trade-offs knowingly accepted:**

Standard lacks Premium's managed WAF rule sets, managed bot protection and Private Link origins. Compensated for by strict input validation, rate limiting, origin restrictions, secure coding and testing.

**Review trigger (when to revisit this):**

Upgrade when managed OWASP/bot rules become required, custom-rule maintenance gets excessive, attack volume/sophistication outgrows compensating controls, compliance demands managed protection, or private origins become mandatory.

## ADR-002 — West Europe active, Sweden Central warm standby (not active-active, not cold standby)

**Why this decision was needed:**

Meet the recovery objective (RTO ≤60 min, RPO ≤15 min) without the cost and complexity of two regions both taking live writes.

**Alternatives considered and their fate:**

- Single region — rejected: can't meet the recovery objective at all.
- Active-active writes — deferred: adds conflict handling, consistency and testing cost not justified yet.
- Cold standby — rejected: provisioning during a live incident makes RTO too unpredictable.

**Trade-offs knowingly accepted:**

Recovery isn't instant — SQL promotion, dependency validation and traffic activation must all complete first. Standby capacity costs money even while serving no traffic. Async replication can lose recent writes, so reconciliation is mandatory after any failover.

**Review trigger (when to revisit this):**

Revisit active-active only when regional traffic becomes continuous and business-critical, the tested RTO can't satisfy the business, or Nordic expansion needs both regions serving normal traffic simultaneously.

## ADR-003 — Azure App Service (not AKS, not VMs, not Functions as the core platform)

**Why this decision was needed:**

The workload is a conventional, continuously-available web/API platform — not something that needs container orchestration or event-driven functions as its primary shape.

**Alternatives considered and their fate:**

- AKS — rejected for Release 1: orchestration and security-operating burden is disproportionate to the workload.
- Virtual machines — rejected: patching, clustering, load balancing and OS operations are all avoidable extra work.
- Azure Functions (as the main platform) — rejected: the core system isn't primarily event-driven.

**Trade-offs knowingly accepted:**

App Service platform constraints (e.g. shared-plan behaviour, hosting model limits) apply everywhere.

**Review trigger (when to revisit this):**

Reassess only if a workload genuinely needs container orchestration, unsupported runtimes, specialized networking, sustained worker processes, or portability that App Service can't provide.

## ADR-004 — Shared regional P1v3 plans; Sweden Central raised to 2 standing workers (V6 change)

**Why this decision was needed:**

Efficient initial capacity with applications kept separately deployable, and — after a V6 review — a DR failover with no scale-up delay in the critical path.

**Alternatives considered and their fate:**

- The original V5 decision kept Sweden Central at just 1 warm worker, scaling to 2 only after a disaster was declared. Review found this made the standby's very first response to a real incident an untested scale-up event, right when resilience matters most — so V6 raises the floor to a standing 2, matching West Europe.

**Trade-offs knowingly accepted:**

Applications share regional CPU/memory, so one noisy workload can affect the others — per-app telemetry, limits, health checks and capacity alerts are required. Deployment slots don't add compute capacity. Sweden Central now carries standing compute cost outside a declared incident — treated as the price of a delay-free failover, not idle spend. Cost impact: +DKK 1,450/month.

**Review trigger (when to revisit this):**

Move the API (or another workload) to its own dedicated plan once it repeatedly exceeds 60% of shared capacity, causes contention, or needs independent scaling/isolation.

## ADR-005 — Azure SQL Database + failover group, zone-redundant primary (V6 change)

**Why this decision was needed:**

A managed relational platform with controlled regional recovery, plus — after a V6 review — real zone-level resilience for the primary database.

**Alternatives considered and their fate:**

- SQL Server on VMs — rejected: adds OS, patching, backup and HA management.
- Cosmos DB — rejected: the transactional relational domain and migration source both fit SQL.
- Active-active writes — rejected: conflict resolution/consistency complexity not justified.
- Automatic uncontrolled failover — rejected initially: business dependencies and possible data loss need human assessment first.
- The original V5 decision left the West Europe primary without zone redundancy, so the stated 99.9% availability target rested on a single-zone database even though the DR story covered whole-region loss. V6 closes this gap directly.

**Trade-offs knowingly accepted:**

A forced failover may still lose recent transactions; reconciliation of payments, orders, inventory and webhooks is required afterward. Cost impact: +DKK 200/month.

**Review trigger (when to revisit this):**

Resize or change tier after query tuning once measured CPU, IO, latency, storage or concurrency crosses an agreed threshold. Redesign only if the relational model or recovery requirement changes materially.

## ADR-006 — Private production data services (SQL, Blob, Key Vault, Azure OpenAI)

**Why this decision was needed:**

Prevent any direct public access to production data services.

**Alternatives considered and their fate:**

- The alternative — leaving any of these publicly reachable — was never seriously on the table given the requirements; the decision is about how private access is implemented, not whether.

**Trade-offs knowingly accepted:**

DNS, VNet integration and deployment sequencing become critical dependencies. Private connectivity never grants authorization by itself — Entra identity, RBAC and SQL permissions are still required on top. Tests must prove private resolution and access before public access is disabled.

**Review trigger (when to revisit this):**

Reassess network topology if centralized egress, inspection, hub-and-spoke connectivity, or more private services become necessary.

## ADR-007 — The API is the only production data-access boundary

**Why this decision was needed:**

Give authorization, auditing, throttling and business rules exactly one enforcement point.

**Alternatives considered and their fate:**

- Letting portals or mobile clients connect directly to SQL/Blob — rejected: breaks the single point of authorization and auditability that a multi-tenant marketplace needs.

**Trade-offs knowingly accepted:**

The API becomes a genuinely critical dependency — it must scale, fail safely, and expose useful health signals. IDOR/BOLA and cross-vendor negative tests become mandatory release gates, not nice-to-haves.

**Review trigger (when to revisit this):**

—

## ADR-008 — Separate identity systems for customers, vendors, employees and workloads

**Why this decision was needed:**

Different actors need different lifecycle, risk and assurance handling — mixing them creates governance problems.

**Alternatives considered and their fate:**

- One shared identity model for everyone — rejected: lifecycle, risk and authorization requirements genuinely differ by actor type.
- Locally stored application passwords — rejected: avoidable credential and recovery risk.
- Long-lived workload secrets — rejected wherever managed identity or federation is available instead.

**Trade-offs knowingly accepted:**

More identity governance surface area to manage (more groups, more review cycles).

**Review trigger (when to revisit this):**

Review tenant structure and identity governance when vendor onboarding volume, partner federation, regulatory obligations, or customer-identity needs change materially.

## ADR-009 — Managed identities + regional Key Vaults (not shared secrets)

**Why this decision was needed:**

Minimize stored secrets and keep provider credentials that can't avoid being secrets tightly scoped and regionally independent.

**Alternatives considered and their fate:**

- Sharing one Key Vault or secret set across regions — rejected: breaks regional independence and DR realism.

**Trade-offs knowingly accepted:**

Role assignments and SQL contained users become deployment dependencies that must be built correctly. Provider secrets that remain still need rotation and incident procedures. Key Vault is not automatically replicated cross-region — required DR secrets are provisioned deliberately and tested quarterly.

**Review trigger (when to revisit this):**

—

## ADR-010 — Bicep for infrastructure as code (not Terraform, not portal clicking)

**Why this decision was needed:**

Azure-native, repeatable delivery, matched to current team skills.

**Alternatives considered and their fate:**

- Terraform — deferred: offers multi-cloud portability the project doesn't need yet, and adds a second toolchain/state model.
- Portal-only deployment — rejected: not repeatable, reviewable, or suitable for building DR parity.
- Hand-authored ARM JSON — rejected: Bicep gives a clearer Azure-native authoring model for the same result.

**Trade-offs knowingly accepted:**

The implementation is inherently Azure-specific.

**Review trigger (when to revisit this):**

Reassess Terraform only for a genuine multi-cloud requirement, an organization-wide standard, or if the team accepts the migration and state-management cost.

## ADR-011 — GitHub Actions with OIDC workload federation (no stored Azure secrets)

**Why this decision was needed:**

Short-lived deployment authentication with nothing long-lived to leak.

**Alternatives considered and their fate:**

- A stored Azure client secret in GitHub — rejected: creates a long-lived credential that's a standing risk.
- Personal Azure credentials — rejected: deployment can't depend on one person's account.
- Self-hosted VNet runner — deferred until a private SCM or source-IP-restricted deployment becomes mandatory.

**Trade-offs knowingly accepted:**

OIDC trust configuration and GitHub environment protection become security-critical themselves. Workflows are scoped to only what they need. CI/CD can deploy infrastructure and applications, but can never authorize or activate DR traffic.

**Review trigger (when to revisit this):**

—

## ADR-012 — SQL transactional outbox before Service Bus

**Why this decision was needed:**

Reliable local transactions without adding message-broker complexity before there's evidence it's needed.

**Alternatives considered and their fate:**

- Service Bus from day one — rejected for Release 1: adds operational surface area the current workload doesn't yet justify.

**Trade-offs knowingly accepted:**

Outbox throughput is coupled to SQL, and offers less independent scaling than a brokered design would.

**Review trigger (when to revisit this):**

Introduce Service Bus when event volume, fan-out, independent scaling, cross-service decoupling, delivery semantics, or operational isolation needs exceed what the SQL outbox design can do.

## ADR-013 — Azure-native observability before Microsoft Sentinel

**Why this decision was needed:**

Get required observability at a cost proportionate to current scale, without buying a SIEM/SOC capability that isn't staffed yet.

**Alternatives considered and their fate:**

- Microsoft Sentinel from day one — rejected for Release 1: not needed until formal SOC/SIEM workflows exist.

**Trade-offs knowingly accepted:**

This is explicitly not a full SOC or enterprise SIEM capability — Operations owns dashboards, alert tuning, retention and runbooks directly.

**Review trigger (when to revisit this):**

Adopt Sentinel when formal SOC workflows, cross-source correlation, advanced threat analytics, security automation, or a regulatory SIEM requirement justifies the licensing, ingestion and staffing cost.

## ADR-014 — Employee-only, West Europe-only Operations Assistant

**Why this decision was needed:**

Bounded employee value from AI, without making AI part of the disaster-recovery critical path.

**Alternatives considered and their fate:**

- Customer-facing generative AI — out of scope: adds product-safety and privacy requirements unrelated to the operational use case.
- A second AI deployment in Sweden Central — deferred: AI is explicitly non-critical during DR.
- Autonomous remediation — rejected: operational changes need deterministic automation and human authorization, not a model deciding.

**Trade-offs knowingly accepted:**

AI is unavailable during a West Europe regional outage — accepted, because it isn't required for commerce or recovery.

**Review trigger (when to revisit this):**

Add regional AI resilience only if the assistant becomes operationally mandatory during DR. Any write or autonomous capability needs its own separate security, safety and architecture decision.

## ADR-015 — DKK 15,000 baseline / DKK 16,500 authorized budget

**Why this decision was needed:**

Align funding with the design that was actually engineered and priced, not the early placeholder guess.

**Alternatives considered and their fate:**

- Keeping the old DKK 10,000 ceiling — rejected: it predates the final compute, SQL geo-secondary, WAF, private connectivity, security monitoring and warm-standby requirements being reconciled.
- A cost-optimized DKK 9,000–10,500 launch — noted as a separate, smaller design; explicitly cannot silently replace the approved target.

**Trade-offs knowingly accepted:**

Requires reforecasting as consumption grows; monthly budget/forecast/variance review becomes mandatory.

**Review trigger (when to revisit this):**

Reforecast before deployment, before cutover, and monthly after launch; also whenever sustained load changes compute, SQL, telemetry, security tier, egress or DR capacity.

## ADR-016 — Human-authorized disaster recovery only

**Why this decision was needed:**

Prevent unsafe or premature database promotion and traffic activation.

**Alternatives considered and their fate:**

- Letting CI/CD or an automated trigger declare a disaster and activate DR — rejected: a wrong automatic call could promote a database with unassessed data loss, or fail over when the primary would have recovered on its own.

**Trade-offs knowingly accepted:**

Manual control reduces the risk of an unsafe failover, but adds time to recovery — accepted, because the tested RTO (≤60 min) already accounts for the human decision step. Twice-yearly exercises must prove the RTO/RPO targets are actually met with this human step included.

**Review trigger (when to revisit this):**

If the RTO/RPO targets repeatedly fail in exercises, the business must fund automation/capacity changes or formally revise the objectives — the target doesn't just quietly slip.

# 8. Security — How the Design Defends Itself

Security approval for this project is conditional, not automatic: production cutover is explicitly blocked while any critical or high-risk finding, untested recovery access, public data endpoint, shared administrator account, stored Azure deployment secret, or unverified vendor isolation remains open. The main residual risks the team accepted going in are the limited WAF capability of Front Door Standard, heavy reliance on application-level authorization in a multi-tenant marketplace, shared App Service compute, third-party provider exposure, asynchronous regional replication, and no continuously staffed security operations function.

## 8.1 The security model in one paragraph

Zero-trust, defense-in-depth: verify every user, workload and deployment identity; grant the minimum permission for the minimum time; treat every network, device, token, payload and external integration as potentially hostile; require both connectivity and authorization (neither substitutes for the other); keep the Nordic API as the only business/data boundary; expose applications through Front Door only; build everything through Bicep and protected pipelines to avoid configuration drift; collect enough evidence to actually detect, investigate and recover from an attack; keep the same posture during DR as during normal operation; and keep the AI assistant read-only, isolated, and non-critical.

## 8.2 Data classification

| Class | Examples | Minimum handling |
|---|---|---|
| Public | Product catalogue, public prices, marketing content | Integrity controls; approved caching and publishing |
| Internal | Runbooks, architecture docs, non-sensitive metrics | Workforce access only; no public sharing by default |
| Confidential | Orders, vendor commercial data, support records, employee data | Need-to-know RBAC, private storage, encryption, audit logging |
| Restricted | Reset material, identity claims, tokens, secrets, payment references | Never logged in raw form; Key Vault/identity platform only; strict retention and access review |

## 8.3 Identity boundaries

| Actor | Identity system | Key controls |
|---|---|---|
| Customers | Entra External ID (external tenant) | Secure sign-up/sign-in, verified recovery, risk monitoring, rate limiting |
| Vendors | Workforce tenant B2B guests | Invitation approval, MFA, vendor membership, quarterly review |
| Employees | Workforce Entra ID | MFA, Conditional Access, device/risk controls where licensed, group-based roles |
| Workloads | System-assigned managed identity | No stored service credentials; minimum data-plane roles |
| GitHub Actions | Federated workload identity | Repository/branch/environment-bound trust; short-lived tokens |
| Emergency admins | Two dedicated cloud-only accounts | Strong MFA, monitored use, sealed recovery process |

The API validates token signature, issuer, audience, lifetime and required scopes on every request, then separately enforces application role, vendor membership, resource ownership and action-level permission. Vendor isolation is treated as a critical control: every vendor-owned record carries an authoritative vendor

identifier, and the API derives the caller's allowed vendor context from trusted server-side claims or membership data — never from a client-supplied identifier alone. Cross-vendor read/update/export/order-action attempts are a mandatory, automated negative-test category, not an afterthought.

## 8.4 Edge and origin protection

Every App Service allows the AzureFrontDoor.Backend service tag only when the request also carries the exact expected X-Azure-FDID header; everything else is denied by default, and the SCM/Kudu endpoint gets an independent, equally strict default-deny rule. Direct azurewebsites.net requests, missing/incorrect Front Door IDs, and unauthorized SCM access must all be provably denied before go-live. This origin check is a network-layer restriction — it is explicitly not a substitute for application authentication or authorization, which still happen on every request regardless.

## 8.5 Threat model — the shape of it

The security strategy document walks 25 concrete attack scenarios (T01–T25) through a consistent chain — threat actor → attack path → prevention → detection → response/recovery → required evidence — grouped into five families:

| Family | Example attacks covered |
|---|---|
| Identity & session (T01–T05) | Credential stuffing, password spray, phishing/token theft, privilege escalation, IDOR/broken access control |
| Edge, availability & network (T06–T11) | DDoS, bot/scraping abuse, direct-origin bypass, DNS hijack, TLS downgrade, private-endpoint misconfiguration |
| Application & API (T12–T17) | Injection, XSS, CSRF, API enumeration/mass assignment, malicious file upload, SSRF |
| Data, secrets & integrations (T18–T22) | Data exfiltration, secret leakage, forged/replayed webhooks, provider compromise, ransomware/destructive deletion |
| Delivery, AI & regional (T23–T25) | Supply-chain/CI-CD compromise, AI prompt injection or data leakage, unsafe or unauthorized regional failover |

Every one of these 25 scenarios also maps back to the 16 business-level risks (S01–S16) from the separate security assessment, and several map further back to a specific current-state risk (e.g. shared-plan contention risk S11 traces to CUR-003, and it's structurally mitigated by ADR-004's V6 standing-capacity change; regional failover data-loss risk S12 traces to CUR-001/CUR-002 and is structurally mitigated by ADR-005's V6 zone-redundancy change). This traceability is deliberate: a control without a rated risk, or a risk without a corresponding control, fails the project's own evidence standard.

## 8.6 Secure delivery pipeline

GitHub Actions authenticates to Azure purely through OIDC — no long-lived Azure client secret is ever stored in GitHub. Federated credentials are restricted by expected issuer, expected Azure audience, exact organization/repository, and protected environment subject, with separate identities for non-production and production. The pipeline can deploy approved resources, but it cannot grant itself roles, read application secrets unnecessarily, declare a disaster, or activate Front Door DR origins — that separation of powers is enforced structurally, not just by policy.

The release path is: pull request → review + scans (dependency, secret, SAST, Bicep lint/validate/what-if) → build one immutable artifact → deploy to staging slots in both regions → warm-up and smoke tests → human approval on evidence → production slot swap → post-deployment validation. The same tested artifact is promoted everywhere — production is never rebuilt from a different source state.

## 8.7 Disaster recovery security

Security controls have to survive regional failure, not just normal operation. The authorized recovery sequence is:

1. Incident commander declares the event and freezes changes.
2. Operations verifies replication state; the security owner approves promotion.
3. The SQL secondary is promoted using named privileged access.
4. DR DNS resolution, secrets, certificates, providers and monitoring are validated.
5. Sweden Central compute is confirmed healthy at its standing capacity (no scale-up wait, per the V6 change).
6. Smoke tests validate authentication, authorization, data access and provider calls.
7. An authorized operator enables the matching Front Door DR origins.
8. Operations monitors, reconciles data, and records every decision made.

CI/CD cannot activate DR under any circumstance. Emergency access must never bypass MFA, authorization, audit, or origin protection. Failback is a new, separately approved change — it is never performed automatically just because West Europe becomes available again.

## 8.8 Go-live security gates (summary)

Before security signs off on production, there must be evidence that: all domains use HTTPS through Front Door; WAF is in prevention mode with tested rate limits; direct-origin and unauthorized SCM requests are denied; SQL/Blob/Key Vault/Azure OpenAI public access is disabled; private DNS resolves correctly from each region; only approved identities have data-plane access; cross-customer, cross-vendor and privilege-escalation tests fail safely; MFA is enforced for employees and vendors; GitHub uses OIDC with no long-lived secrets; provider webhook signature/replay/duplicate tests pass; logs and AI inputs contain no secrets or prohibited personal data; critical alerts reach both a primary and secondary responder; restore and DR exercises meet RPO/RTO without disabling any control; and no critical/high finding remains unresolved.

# 9. Migration Strategy — Getting There Without Breaking the Business

## 9.1 The chosen approach

| Approach | Decision | Reason |
|---|---|---|
| Direct server-for-server lift-and-shift | Rejected | Retains current operational, scaling and recovery limitations |
| Full application rewrite / microservices conversion | Deferred | Adds avoidable delivery risk; not required for Release 1 |
| One-step big-bang migration without rehearsal | Rejected | Data, identity, provider and rollback risks are too high |
| Phased re-platform with two rehearsals | Approved | Preserves business capability while proving repeatability and rollback |
| Permanent hybrid operation | Rejected as an end state | Temporary coexistence only, for migration and stabilization |

The application stays a modular monolith throughout. Changes are limited to what's genuinely required for Linux App Service compatibility, stateless execution, managed identity, Azure SQL, Blob Storage, observability, security and safe deployment — this is a re-platform, not a rewrite.

## 9.2 Twelve migration principles

1. Protect business continuity and authoritative data before migration speed.
2. Discover and measure before selecting detailed migration tooling.
3. Build the destination through Bicep — never hand-build an undocumented production environment.
4. Use immutable application artifacts; promote the same tested artifact between environments.
5. Keep production data private, accessed only through managed identities and least privilege.
6. Rehearse the complete procedure at least twice, at production-like volume.
7. Reconcile technical records and business totals before accepting migrated data.
8. Keep rollback available until the agreed point of no return.
9. Keep application-release rollback separate from data/traffic rollback — they're different procedures.
10. Require named human approval for production cutover, SQL promotion, DR activation and failback.
11. Preserve source backups and evidence until formal decommission approval.
12. Record actual duration, defects, cost and outcomes at every rehearsal and cutover.

## 9.3 The six migration waves

| Wave | Workload | Main treatment | Exit evidence |
|---|---|---|---|
| 0 | Foundation | Build Azure, identity, networking, security, monitoring, CI/CD | Bicep deployment and control tests pass |
| 1 | Non-production applications | Re-platform the four workloads to App Service | Functional, integration and rollback tests pass |
| 2 | Files and database rehearsal copies | Transform and load production-like data | Counts, checksums and business totals reconcile |
| 3 | Identities and integrations | Configure role mappings, providers, secrets, callbacks | Positive and negative authorization/provider tests pass |
| 4 | Production migration | Final data sync and controlled traffic | Business and technical validation |

| Wave | Workload | Main treatment | Exit evidence |
|---|---|---|---|
| | | cutover | accepted |
| 5 | Stabilization and retirement | Monitor, reconcile, retain source read-only, then decommission | Stabilization and decommission approvals signed |

## 9.4 Database migration sequence

1. Inventory schemas, objects, users, jobs, dependencies and unsupported SQL Server features.
2. Profile and cleanse source data; agree transformation and rejection rules.
3. Write version-controlled schema and data-migration scripts.
4. Deploy the target schema and least-privilege database roles.
5. Load a production-like rehearsal copy through the selected secure path.
6. Validate row counts, checksums, referential integrity and business totals.
7. Measure duration and test incremental/final synchronization.
8. At cutover, stop or control source writes at the approved checkpoint.
9. Take a final protected backup and record the source checkpoint.
10. Run final synchronization, repeat reconciliation, and get Data and Business owner sign-off.
11. Enable Azure production writes only after that approval.

> *The exact database migration tool is deliberately not fixed in advance — it's selected after discovery, based on SQL Server version/features, database size, network throughput, encryption, allowed downtime, and whether incremental sync is required. The strategy refuses to assume a tool without evidence.*

## 9.5 Two mandatory rehearsals

**Rehearsal 1 — procedure validation:**

Proves scripts, access, sequencing and initial time estimates; surfaces compatibility, data-quality and integration failures; tests technical rollback; produces the first measured timeline. Every manual step, defect and exception gets recorded, and the runbook/automation is corrected before Rehearsal 2.

**Rehearsal 2 — production-readiness simulation:**

Uses final production-like volume and the actual intended cutover team; runs the full communications/freeze/backup/sync/validation/traffic/rollback sequence end to end; proves the outage fits the approved business window; confirms the point of no return and who has authority to call it. It only passes with no unresolved data loss or corruption, all critical journeys working, reconciliation thresholds met, and a timed rollback executed within the approved window.

> *A third rehearsal is required if anything material changes after Rehearsal 2 (script, schema, identity, provider, volume, topology) or if Rehearsal 2's gate simply doesn't pass.*

## 9.6 Rollback — a business decision, not a button

Before cutover, the Business and Data owners agree the exact “point of no return” — the moment after which reverting to on-premises risks losing or duplicating accepted Azure transactions. Until that point, Azure writes must be prevented or captured in a tested, reversible mechanism. Critically: DNS reversal alone is never treated as a valid rollback, because it can create split-brain data and duplicate payments or orders. If meaningful production writes have already happened in Azure, rollback requires an approved reverse-reconciliation or transaction-replay procedure, not just pointing DNS back.

# 10. Cost Model — Every Kroner Explained

This is a planning estimate, not a Microsoft quotation — real invoices depend on regional prices, contractual discounts, currency conversion and actual measured consumption. The configuration must be recreated in the Azure Pricing Calculator before procurement and refreshed again before production cutover. But every line below maps to a specific, named piece of the architecture — nothing here is a rounded guess.

## 10.1 Release 1 monthly Azure estimate, line by line

| Cost area | Configuration priced | Monthly estimate |
|---|---|---|
| Primary application hosting | 2 × P1v3 workers, West Europe | DKK 2,900 |
| Warm-standby hosting | 2 × P1v3 workers, Sweden Central (V6: raised from 1) | DKK 2,900 |
| Azure SQL primary | GP 2-vCore, zone-redundant (V6 addition), storage + backup | DKK 2,600 |
| Azure SQL geo-secondary | Equal-size secondary, storage + replication | DKK 2,400 |
| Front Door Standard + WAF | Profile, 4 routes/origin groups, WAF policy, requests, transfer | DKK 650 |
| Storage and data protection | 2 accounts, capacity, versioning, soft delete, replication, backup | DKK 350 |
| Monitoring and alerting | App Insights, Log Analytics, diagnostics, alerts, retention | DKK 700 |
| Private connectivity | 7 private endpoints and routine processing | DKK 350 |
| Key Vault and DNS | 2 vaults, operations, certificates, DNS zones | DKK 50 |
| Defender and malware scanning | Selected protection + bounded Blob scan volume | DKK 350 |
| AI Operations Assistant | Pay-as-you-go inference, application-enforced cap | DKK 300 |
| Bandwidth and regional transfer | Normal internet responses + replication traffic | DKK 350 |
| Minor platform services | Public DNS, delivery, control-plane allowance | DKK 50 |
| Estimated Azure operating subtotal | | DKK 13,950 |
| Planning contingency (~7.5%) | Usage/telemetry/currency/price variance | DKK 1,050 |
| Normal-operation planning baseline | | DKK 15,000 |

> *The V6 change note: raising Sweden Central to a standing 2 workers and making the West Europe SQL primary zone-redundant together add DKK 1,650/month (DKK 1,450 for the extra standby worker + DKK 200 for zone redundancy) over the V5 baseline. This closes the two largest previously-disclosed resilience gaps — a single-instance DR standby, and a single-zone primary database — while staying inside the newly authorized DKK 16,500 envelope.*

## 10.2 Normal, peak and DR scenarios

| Scenario | Expected monthly cost | Approval position |
|---|---|---|
| Lower-usage normal month | DKK 12,000–13,000 | Within baseline |
| Planned normal month | DKK 15,000 | Baseline |
| Upper normal range | DKK 14,000–15,000 | Within authorization; investigate variance |
| Seasonal campaign month | DKK 15,500–18,000 | Temporary approval required |

| Scenario | Expected monthly cost | Approval position |
|---|---|---|
| Full-month DR operation | DKK 16,300–18,800 | Incident funding and reforecast required |

## 10.3 Where the budget goes as the business grows

| Stage | Architecture position | Monthly estimate | Annualized |
|---|---|---|---|
| Release 1 | Shared regional plans, 2-vCore SQL pair, warm standby | DKK 15,000 | DKK 180,000 |
| Year 1 upper envelope | Normal variance within authorization | Up to DKK 16,500 | Up to DKK 198,000 |
| Year 2 growth | More compute/SQL capacity, higher telemetry/traffic | DKK 16,000–21,000 | DKK 192,000–252,000 |
| Three-year target | Dedicated API capacity, larger data tier, integration/security growth | DKK 22,000–28,000 | DKK 264,000–336,000 |
| Five-year Nordic expansion | More independent plans, larger data platform, mature security ops | DKK 38,000–55,000 | DKK 456,000–660,000 |

Growth doesn't create a smooth cost curve — the bill rises in discrete steps, each time an additional plan, database tier, or security service is approved. Deferred capabilities each have an order-of-magnitude cost estimate ready for when their trigger fires: Front Door Premium (+DKK 1,800–3,000/mo), separate API plans (+DKK 3,000–5,000/mo), API Management (+DKK 1,500–6,000/mo), Service Bus (+DKK 200–1,000/mo), managed Redis-compatible cache (+DKK 500–2,000/mo), Microsoft Sentinel (+DKK 1,000–5,000+/mo), a Sweden Central AI deployment (+DKK 100–700/mo), and an active-active operating model (+DKK 5,000–15,000+/mo).

## 10.4 Budget alerting

| Threshold (of DKK 16,500) | Amount | Required action |
|---|---|---|
| 50% actual | DKK 8,250 | Review month-to-date actual and forecast |
| 70% actual | DKK 11,550 | Investigate compute, SQL, logs and bandwidth |
| 85% actual | DKK 14,025 | Record corrective action or approved peak |
| 95% actual | DKK 15,675 | Escalate to finance and sponsor |
| 100% actual | DKK 16,500 | Require approval for nonessential paid changes |
| 90% forecast | DKK 14,850 | Early warning — investigate trend before it becomes an actual breach |
| 110% forecast | DKK 18,150 | Formal capacity and budget review |

> *Budget alerts are advisory and never automatically stop production. And a hard rule the project holds itself to: never quietly remove WAF, private access, the SQL secondary, recovery replication, or essential monitoring just to hit an old budget number.*

# 11. The Diagram Set — What Each Picture Shows

Alongside the ten documents, the project maintains nine architecture diagrams (.drawio format), each explicitly cross-referenced to 04-target-architecture.md, dated 4 August 2026. They are the visual backbone of everything described in Section 6, and they're worth opening side-by-side with this report once you start building. Here's what each one shows and why it exists:

### 01 — Architecture Overview

The executive/portfolio view: customers/vendors/employees → Entra identity → Front Door → West Europe (active) and Sweden Central (warm standby), each with its App Services, private SQL/Blob/Key Vault, and the AI Operations Assistant. Shows async SQL replication and “DR origins disabled during normal operation.” Good starting point for anyone new to the project.

### 02 — Identity and Traffic Flow

The exact eight-step request path: DNS resolves → user authenticates → Entra returns token → client sends HTTPS request → WAF inspects → Front Door selects route/origin → portal calls the API → API validates token/role/authorization. Also shows that mobile clients call the API hostname directly, without going through a web portal.

### 03 — Regional Network and Data

Zooms into one region's private-access pattern: VNet integration → private DNS resolution → private endpoint → service authorization → data, for SQL, Blob, Key Vault and (WEU-only) Azure OpenAI. Carries the explicit reminder that a private endpoint is network reachability, not permission.

### 04 — Deployment and Operations

The nine-step release pipeline (PR → build/test/scan → OIDC token exchange → Bicep validate/deploy → staging deploy to both regions → warm-up/smoke test → approval → slot swap → telemetry verification), plus the governance layer (policy, budgets, tags) sitting alongside it.

### 05 — Disaster Recovery

The ten-step failover order: declare incident → assess replication → promote SWC SQL → validate dependencies → verify standing capacity → enable DR origins → smoke test → serve/monitor → reconcile → plan failback. States the guardrails plainly: failover is human-authorized, GitHub Actions cannot activate DR traffic, and RPO is a target, not a guarantee.

### 06 — Full Security Architecture

The most detailed diagram: five layers from users/attack-surface through identity/edge, application/authorization, private network/data, to secure delivery/detection/response. States the control principle explicitly — a request is only allowed when network reachability, identity, authorization, data scope and monitoring all succeed together.

### 07 — Current On-Premises Architecture

The as-is picture: one single production location, Windows-based hosting, the four current applications, and the external dependencies (payment, email, SMS). Marked up with the current-state risks and evidence gaps discussed in Section 3.

### 08 — Migration and Cutover Flow

Three stages — discover & build, prove repeatability & readiness, controlled production cutover — ending in an explicit GO / NO-GO decision diamond, with a documented rollback path for the NO-GO branch and the point-of-no-return guardrail.

### 09 — Monitoring and Incident Response Flow

Four stages from raw telemetry through centralization/correlation/detection, to classification/routing/ownership (critical/high/medium/low), to full incident response, recovery and learning — with the AI assistant's bounded role (summarize only, cannot contain/remediate/close/activate DR) shown explicitly.

# 12. The Build Order — What We Build, In What Sequence

This is the answer to "what do we build after what." The roadmap is deliberately gate-driven: finishing a task, or even successfully deploying an Azure resource, does not automatically unlock the next phase. Each phase must produce real evidence and get a named approval before the next, riskier step begins. The 16 weeks below only start counting after this document set is approved — roughly ten weeks of planning (26 May – 4 August 2026) already produced everything described in Sections 1–11.

## 12.1 The nine phases, at a glance

| Phase | Weeks | Main outcome | Exit gate |
|---|---|---|---|
| 0. Mobilization | 1 | Scope, ownership and delivery controls approved | Project kickoff gate |
| 1. Engineering foundation | 2–3 | Repository, Bicep structure, environments and OIDC ready | Foundation gate |
| 2. Core Azure platform | 4–5 | Network, identity, data and shared services deployed | Platform gate |
| 3. Application and CI/CD | 6–8 | Four workloads deploy safely through staging | Release gate |
| 4. Security and observability | 9–10 | Controls, logs, alerts and dashboards validated | Security gate |
| 5. Migration rehearsal | 11–12 | Repeatable data and application migration proven | Migration-readiness gate |
| 6. DR and operational readiness | 13 | Failover, recovery and runbooks tested | Operational gate |
| 7. Production cutover | 14 | Controlled migration and traffic activation | Go-live gate |
| 8. Stabilization and optimization | 15–16 | Service stabilized, tuned and handed over | Project-closure gate |

## Phase 0 — Mobilization (Week 1)

**What gets built / done:**

- Confirm scope, assumptions, exclusions and success measures.
- Approve the DKK 16,500 authorized monthly operating envelope and cost-alert recipients.
- Name the business sponsor, product owner, platform owner, security owner, data owner, application owner, incident commander and DR owner.
- Confirm Azure tenant, subscriptions, billing scope, GitHub organization and domain ownership.
- Establish change control, risk review, decision logging and weekly reporting.
- Freeze superseded documents and adopt the current document set as the implementation baseline.
- Confirm third-party provider contacts, test environments and migration blackout periods.

**Exit criteria (what must be true before moving on):**

Required owners accept responsibility; funding and access (Azure, GitHub, test providers) are available; every critical assumption has an owner and a validation date; no unresolved scope issue blocks foundation work.

## Phase 1 — Engineering foundation (Weeks 2–3)

**What gets built / done:**

- Create the repository structure: infra/, application folders, workflows, tests, operational docs.
- Build reusable Bicep modules and environment parameter files.
- Implement naming, tagging, resource-group and diagnostic-setting conventions.
- Configure GitHub branch protection, CODEOWNERS and protected environments.
- Create separate federated identities for infrastructure deployment and application deployment.
- Restrict OIDC trust by organization, repository, branch or protected environment.
- Add Bicep linting, validation, what-if, unit tests, dependency scanning and secret scanning.
- Define dev/test/production configuration handling without ever committing secrets.

**Exit criteria (what must be true before moving on):**

A test deployment authenticates via OIDC and deploys only within its approved scope; pull requests can't bypass required review/checks; Bicep validation and what-if both succeed; secret scanning confirms nothing is stored in the repo.

## Phase 2 — Core Azure platform (Weeks 4–5)

**What gets built / done:**

- Deploy shared resource groups, tags, budgets, policies and diagnostic settings.
- Deploy West Europe and Sweden Central VNets with separate integration and private-endpoint subnets.
- Deploy regional App Service plans, applications and staging slots.
- Deploy Azure SQL primary, geo-secondary and failover group.
- Deploy regional Storage accounts and approved critical-blob object replication.
- Deploy independent regional Key Vaults and controlled DR secret provisioning.
- Deploy West Europe Azure OpenAI and its private endpoint.
- Configure Private DNS: four service zones in West Europe, three in Sweden Central.
- Deploy Front Door, custom domains, routes, origin groups, health probes and WAF policy.
- Configure managed identities, minimum data-plane access and App Service origin restrictions.

**Exit criteria (what must be true before moving on):**

Bicep deployment is repeatable with no unexplained drift; SQL/Storage/Key Vault/Azure OpenAI public access is disabled as designed; the API resolves approved services to private IPs; portals have no direct data-service permissions; direct-origin requests are denied while valid Front Door traffic succeeds; actual platform cost stays within tolerance.

## Phase 3 — Application modernization and CI/CD (Weeks 6–8)

**What gets built / done:**

- Externalize application configuration; remove local durable state.
- Implement health endpoints, structured logging, correlation IDs and dependency telemetry.
- Integrate External ID, B2B and workforce authentication.
- Implement API-side role, action, ownership and vendor-isolation authorization.
- Replace data-service credentials with managed identity.
- Implement parameterized SQL access, safe error handling, idempotency and provider resilience.
- Implement signed webhook validation, timestamp checks, replay prevention and reconciliation.

- Implement upload quarantine, malware scanning and promotion to trusted storage.
- Build immutable artifacts and deploy them to both regional staging slots.
- Run warm-up and smoke tests; require human approval before production slot swap.
- Keep Sweden Central application versions aligned without activating DR traffic.

**Exit criteria (what must be true before moving on):**

The same artifact deploys successfully to both regions; staging tests pass before production approval is even possible; cross-vendor and unauthorized-access tests fail securely; slot swap and compatible swap-back are demonstrated; CI/CD is proven unable to activate Front Door DR traffic.

## Phase 4 — Security and observability (Weeks 9–10)

**What gets built / done:**

- Tune WAF custom rules in detection mode, then move approved rules to prevention.
- Enforce MFA, Conditional Access, privileged access and group-based RBAC.
- Enable application, Front Door, Entra, SQL, Key Vault, Storage, Activity Log and deployment telemetry.
- Configure Application Insights, Log Analytics, Azure Monitor alerts, Action Groups and workbooks.
- Configure Defender for Cloud recommendations for the production risk profile.
- Run SAST, dependency, secret, infrastructure and dynamic application security tests.
- Test credential stuffing, origin bypass, IDOR/BOLA, SQL injection, XSS, CSRF, API abuse, forged webhooks, malicious uploads, excessive privilege.
- Validate incident alert routing, evidence retention and responder acknowledgement.
- Correct or formally accept every open security finding before go-live.

**Exit criteria (what must be true before moving on):**

No unresolved critical/high finding without executive acceptance; required logs are searchable with correlation data and no exposed secrets; critical alerts reach both primary and secondary responders; direct-origin, cross-vendor and unauthorized data-access tests are blocked; WAF prevention meets agreed false-positive and coverage thresholds.

## Phase 5 — Migration rehearsal (Weeks 11–12)

**What gets built / done:**

- Clean and map source data to the target schema.
- Define full-load and delta-load procedures with encryption and restricted staging.
- Rehearse database migration in a production-like environment.
- Rehearse approved Blob migration; verify checksums, counts and metadata.
- Test identity migration / account transition procedures.
- Execute application functional, integration, performance and user-acceptance tests.
- Measure migration duration, final synchronization window and expected downtime.
- Rehearse rollback to the on-premises platform.
- Finalize customer, vendor, employee and support communications.

**Exit criteria (what must be true before moving on):**

Two successful rehearsals complete with no unresolved data loss or corruption; reconciliation meets the approved accuracy threshold; performance meets the agreed baseline under production-like load; the measured outage fits the approved business window; rollback remains achievable within the decision window.

## Phase 6 — DR and operational readiness (Week 13)

**What gets built / done:**

- Simulate a West Europe outage and declare an incident.
- Confirm human authorization before DR activation.
- Promote the SQL secondary, validate Blob availability, confirm regional secrets.
- Verify Sweden Central's standing two-worker capacity and application health before activating DR traffic.
- Validate identity, private DNS, permissions, provider access and security controls.
- Activate Front Door standby origins and execute business smoke tests.
- Measure achieved RPO and RTO.
- Reconcile data and test controlled failback.
- Run incident, security, backup-restore and operational tabletop exercises.
- Finalize runbooks, escalation contacts, dashboards and support handover.

**Exit criteria (what must be true before moving on):**

RPO ≤ 15 minutes and RTO ≤ 60 minutes, or the business formally accepts revised objectives; CI/CD is proven unable to authorize DR traffic activation; backup restore and regional failover are both tested independently (neither substitutes for the other); Operations and Security owners approve production readiness.

## Phase 7 — Production cutover (Week 14)

**What gets built / done:**

- Confirm owners, support coverage, backups and the rollback checkpoint.
- Announce the change window; place the source system into the approved write-control state.
- Execute the final data delta and reconciliation.
- Validate identity, API, orders, payments, vendors, administration and provider integrations.
- Enable West Europe production origins and gradually shift public traffic through Front Door.
- Monitor application, dependency, security and business metrics throughout.
- Obtain business confirmation and only then close the rollback window.

**Exit criteria (what must be true before moving on):**

Critical customer and vendor journeys work correctly; data reconciliation is approved by data and business owners; no critical security, availability or financial alert is unresolved; rollback executes if a pre-agreed stop condition is reached; the business sponsor authorizes continued production operation.

## Phase 8 — Stabilization and optimization (Weeks 15–16)

**What gets built / done:**

- Operate enhanced monitoring with daily defect, risk and cost reviews.
- Tune autoscale, SQL capacity, WAF thresholds, logging volume and alert noise.
- Correct deployment, application and operational defects as they surface.
- Validate Azure invoices against the DKK 15,000 baseline and DKK 16,500 approved budget.
- Complete knowledge transfer and assign business-as-usual ownership.
- Enable the read-only AI Operations Assistant — only after monitoring data quality and security gates pass.
- Create the final architecture-as-built record and portfolio demonstration.
- Capture lessons learned and update the improvement backlog.

**Exit criteria (what must be true before moving on):**

Service levels and error rates remain stable for the agreed observation period; actual cost is understood and approved; every operational control has a named long-term owner; remaining risks are recorded, prioritized and accepted; the sponsor signs project closure.

## 12.2 Mandatory delivery gates

| Gate | Required evidence | Approvers |
|---|---|---|
| G1 — Project kickoff | Scope, owners, budget, access and risks | Sponsor, project lead |
| G2 — Foundation | OIDC, protected repository, Bicep validation | Platform, DevOps, security |
| G3 — Platform | Repeatable deployment, private access and cost check | Architecture, platform, security |
| G4 — Release | Staging, smoke tests, authorization tests and rollback | Application, QA, DevOps |
| G5 — Security | Threat tests, access reviews, logging and remediation | Security, risk owner |
| G6 — Migration readiness | Two rehearsals, reconciliation, UAT and rollback | Data, application, business |
| G7 — Operational readiness | DR, restore, monitoring, runbooks and support | Operations, security, DR owner |
| G8 — Go-live | All mandatory gates green and change approved | Sponsor, change authority |
| G9 — Closure | Stable service, cost review, handover and residual risks | Sponsor, operations |

## 12.3 What “project complete” actually means

The transformation is only complete when all of the following are true — diagrams and planning documents alone never satisfy this definition:

- The approved infrastructure and applications are deployed through controlled pipelines.
- Production data is migrated and reconciled.
- Security, performance, recovery and cost gates have all passed.
- Customer, vendor and employee journeys operate successfully.
- Monitoring, incident response, backup, DR and support processes are accepted.
- The as-built architecture and architecture-decision records match reality.
- Residual risks and future improvements have owners and review dates.
- The business sponsor signs the project-closure decision.

# 13. Governance, Roles and the Top Risks to Watch

## 13.1 Who owns what

| Workstream | Accountable owner | Supporting roles |
|---|---|---|
| Scope, value and go-live | Business sponsor | Product owner, project lead |
| Architecture and Bicep | Platform owner | Security and application owners |
| Application modernization | Application owner | Developers, QA, platform owner |
| Identity and access | Identity owner | Security and application owners |
| Data migration | Data owner | Database engineer, application owner |
| Security assurance | Security owner | Identity, platform and application owners |
| CI/CD and releases | DevOps owner | Platform and application owners |
| Monitoring and support | Operations owner | Application, security and platform owners |
| Disaster recovery | DR owner | Incident commander, data and platform owners |
| Cost governance | FinOps owner | Sponsor and platform owner |

> *No individual may silently approve their own unresolved high-risk exception. Production cutover, rollback after the point of no return, disaster declaration, SQL promotion, DR traffic activation and failback all require the named authority in the approved runbook — never a solo call.*

## 13.2 Principal risks and how they're treated

| Risk | Business effect | Treatment |
|---|---|---|
| Application needs more modernization than expected | Delay and extra cost | Early dependency assessment, proof of concept, reapproval before build |
| Shared regional compute creates contention | Slow or unavailable channels | Per-app telemetry, load tests, autoscale, trigger to separate plans |
| Async replication loses recent writes | Order/payment inconsistency | RPO monitoring, idempotency, outbox/reconciliation, authorized failover |
| Identity migration disrupts access | Lost sales or support workload | Pilot cohorts, role mapping, negative tests, rollback procedure |
| Third-party provider failure | Delayed/duplicated commerce activity | Timeouts, retries, signed webhooks, idempotency, reconciliation |
| Security attack or control failure | Fraud, data loss or outage | Defence-in-depth controls, testing, monitoring, response playbooks |
| Migration reconciliation fails | Incorrect production records | Two rehearsals, counts, checksums, business validation, rollback gate |
| Consumption exceeds forecast | Budget breach | Budget alerts, telemetry controls, monthly review, reforecast triggers |
| Team is not operationally ready | Longer incidents, unsafe changes | Runbooks, training, named owners, operational acceptance |
| AI produces unsafe advice or exposes data | Bad decisions or confidentiality breach | Approved sources, redaction, read-only access, audit, human judgment |

## 13.3 Success measures the whole project is judged against

| Measure | Acceptance target |
|---|---|
| Availability | ≥ 99.9% monthly for accepted customer-facing production services |
| Recovery | Timed exercise demonstrates RTO ≤ 60 minutes and RPO ≤ 15 minutes |
| Performance | Approved Release 1 workload and response-time tests pass |
| Security | No unresolved critical finding; high risks remediated or formally accepted |
| Authorization | Customer, vendor, employee and workload tests pass, incl. cross-vendor negative tests |
| Delivery | Repeatable Bicep deployments and controlled slot-based releases demonstrated |
| Migration | Reconciliation, cutover and rollback evidence accepted after rehearsals |
| Operations | Dashboards, alerts, runbooks, routing, ownership and on-call tests accepted |
| Cost | Refreshed normal-month forecast within DKK 16,500 or additional funding approved |
| Handover | Business, technical, security, data and operations owners sign off |

# 14. Glossary of Terms Used Throughout This Project

| Term | Meaning |
|---|---|
| ADR | Architecture Decision Record — a formal write-up of one accepted design choice, its rejected alternatives, trade-offs and review trigger |
| RTO | Recovery Time Objective — how long recovery is allowed to take (≤ 60 minutes here) |
| RPO | Recovery Point Objective — how much recent data loss is tolerable in a failover (≤ 15 minutes here) |
| WEU / SWC | West Europe / Sweden Central — the two Azure regions used (active / warm standby) |
| Warm standby | A DR environment that's deployed and kept current, but not serving live traffic until authorized |
| Front Door | Azure's global Layer-7 entry point — handles TLS, routing, health probes, WAF and inter-region failover |
| WAF | Web Application Firewall — edge-level filtering of malicious/abusive HTTP traffic |
| Managed identity | An Azure-managed credential for a resource, replacing stored passwords/secrets |
| Private endpoint / Private DNS | Mechanism to reach an Azure PaaS service over a private IP instead of the public internet |
| OIDC | OpenID Connect — used here for GitHub Actions to get short-lived Azure access without a stored secret |
| Failover group | Azure SQL feature that groups a primary and secondary database under one connection name for controlled failover |
| Zone-redundant | A resource is spread across multiple physical availability zones within a region, surviving a single zone's failure |
| Transactional outbox | A pattern for reliably sending messages/events as part of a database transaction, without a message broker |
| IDOR / BOLA | Insecure Direct Object Reference / Broken Object-Level Authorization — accessing another user's/vendor's data by manipulating an identifier |
| PIM | Privileged Identity Management — just-in-time, time-bound elevation of administrative access |
| SOC / SIEM | Security Operations Centre / Security Information and Event Management — not part of Release 1 (Microsoft Sentinel is deferred) |
| Point of no return | The agreed moment in a cutover after which reverting to the old system risks losing or duplicating data |
| Slot swap | Deploying to a staging slot, validating it, then swapping it into production with near-zero downtime |

# 15. Closing Note

Everything in this report is a consolidation of the approved document set — nothing here changes or reinterprets the design. If you're about to start coding, the practical reading order is: this report first for the big picture and the “why” behind each decision, then 04-target-architecture.md for exact resource names/SKUs/settings, then 10-architecture-decisions.md whenever you want the deeper context behind a specific choice, and 09-project-roadmap.md to track which phase gate you're working toward next.

The single most important discipline running through the whole project is this: nothing is treated as done because it was designed, diagrammed, or even deployed — it's only done once there's test evidence behind it, reviewed by a named owner. That discipline is what turns a good architecture diagram into a production system the business can actually trust.